BUILD BRIEF | Independent Reconciliation Audit

What the two research agents got right, missed, overstated and how the master blueprint resolves it

27 October 2026

Audit verdict: MERGE WITH MATERIAL CORRECTIONS. Neither research document should be handed unmodified to a coding agent. The first is stronger on approved requirements, evidence semantics, source security and realistic scope. The second is stronger on hooks, reverse drift, prompt autopsy, Comprehension Map and measured benchmarks. Some assertions in the second are too categorical, and both rely on unverified competition details. The resulting source of truth is **Build_Brief_MASTER_Blueprint.md** (and companion PDF), version 1.0.

1. Inputs and methodology

Document A: previous 21-section *Build Brief - Research and Engineering Blueprint* from October 27, 2026. **Document B:** newly supplied second-agent 10-part *BUILD BRIEF - Research, Validation and Build Plan*, also October 27. Earlier conversation context from October 19 described a prototype concept (intent capture, source-linked reviews, version comparisons and handover exports); current code is **not independently inspected** here. Official Meta, Cerebral Valley, Kiro, GitHub Spec Kit, git-ai and Anthropic/arXiv research were consulted for the most material claims.

Evidence rating: Confirmed = official primary documentation supports the claim; **Provisional** = plausible secondary/hackathon report but official signed rules unavailable; **Proposed** = future design decision; **Hypothesis** = competitive gap or outcome not measured; **Unsupported** = don't ship or pitch as fact.

2. Summary by audit category

Area	A (previous)	B (new agent)	Decision
Product core	Clear, proof-centered	Strong Capture/Verify/Fix/Learn narrative	Combine into approved contract -> evidence -> recheck -> learning.
Intent capture	Canonical user approvals	Hooks + JSON capture	Combine; hooks capture candidate drafts, **never auto-approve**.
Correctness model	Evidence tiers, unknowns, stale revisions	Five status labels, orphan changes	Combine with source/test/runtime grades and bounded negative findings.
New feature depth	Role explanations, correction packs, research	Reverse drift, constraint guard, prompt autopsy, heatmap	Retain all in product inventory; restrict hackathon implementation.
Meta integration	Substantive structured reasoning	Native Muse agent/plugin packaging	Use both, but Muse is showcase adapter, not hard product dependency.

Area	A (previous)	B (new agent)	Decision
Competitor rigor	Notes crowded overlap	Too strong "nobody closes the loop" claims	Correct using current Spec Kit and Muse documentation.
Security	Tenant/source/privacy model	Local-first plus prompt injection	Merge; fix sensitive handling of local JSON and contributor training mode.
Evidence that people learn	Cautious	Anthropic + Explanation Gate studies	Use as rationale, **not evidence this product works**.
Delivery realism	P0 narrow	Four modules in 8 days + 3-minute video	Reduce to one genuinely working loop with limited extras.
Evaluation	CI and citation checks	12-20 case precision/recall	Add held-out controls, adversarial tests and no fake metrics.
Competition facts	Correct caution about signed rules	Several unverified statements as assertions	Read participant waiver; treat weights/video/repo/new-work as unconfirmed.

3. Findings from the NEW research we explicitly keep

A. Reverse-change detection. This closes the other direction of the traceability graph: "the agent changed this, but which approved intention supports it?" An unlinked diff hunk is an **investigation request**, not automatic misconduct. Add path sensitivity and a human disposition, not an automatic revert.

B. Persistent constraint policy. "Do not touch auth" should survive compaction and agent switching. Use explicit current scope and authorization, hook warnings/approval and post-commit checks. A hook cannot block human Git operations or every possible shell write.

C. Prompt Autopsy. Build Brief should know what failed, not merely rewrite the last prompt. Preserve attempts, test logs, revision and constraints. Classify possible causes with uncertainty; never guarantee a suggested prompt fixes anything.

D. Comprehension Map. Excellent visual differentiator **if** it represents actual, consented assessment evidence. `Unmeasured` cannot be scored as weak understanding. Optional AI-line attribution must come from real git-ai/agent provenance, not invented estimates.

E. Benchmark with class-level metrics. Critical because claims about verification accuracy can be tested. Add counterexamples, missing evidence, citation tampering, harmless refactors and independent hold-out cases. Publish false positives and abstentions, not only a green accuracy percentage.

F. Integration-first packaging. Muse's official docs now confirm `UserPromptSubmit`, `SessionStart`, `PreToolUse`, `Stop` lifecycle hooks and MCP/skill support. This makes native prompt capture feasible with the actual installed version. It still does not make MCP alone a universal prompt interceptor. Official: https://dev.meta.ai/docs/muse-code/extending .

4. Important corrections to the NEW research

1. "Nobody closes the loop." Not established. Current GitHub Spec Kit explicitly provides *Implement -> Converge*, recorded verification and issue-assessment flows (https://github.github.com/spec-kit/). Kiro has sophisticated specs, hooks, steering, and testing (https://aws.amazon.com/documentation-overview/kiro/). Muse Code has session journals and verification observers (https://dev.meta.ai/docs/muse-code/extending). **Our distinct hypothesis must be tested**, not asserted as a market fact.

2. "Spec Kit only records specs" and "reviewers never check intent." Both overbroad. Advanced reviewers accept repository guidance, issue context and product requirements in various workflows. Sell continuous *approved business-intent state + proof + teaching*, not generic "code review does not know requirements."

3. "Missing" is a universal verdict. Missing evidence is often **not established**, especially with partial repository capture. Externally supported contradiction and exhaustive-search absence are different. In the master, mark a criterion `not_established` when coverage is inadequate, and record the exact additional evidence required.

4. "Muse Code hooks may lack prompt-submit." Official current docs explicitly list `UserPromptSubmit`, correcting B's own uncertainty. The actual payload/handler syntax must still be checked against current official references and installed CLI. https://dev.meta.ai/docs/muse-code/extending

5. "Store the API key in a settings file." This is incomplete and potentially unsafe. Prefer OS credentials store/protected server environment with restrictive file permissions only as fallback; never put the key into project-controlled files or logs. Additionally, Meta's October 12 policy says **Coding Harness Subscription** may default to Contributor services even when a model is selected. https://dev.meta.ai/legal/geographic-use-policy

6. "Whole repo in 1M context makes retrieval unnecessary." A big context window does not solve proprietary source security, generated folders, outdated code, irrelevant files, token budgets or attention/accuracy problems. Use structural/lexical selection first; a bounded full-repo mode is optional with consent. https://dev.meta.ai/docs/models

7. "17 points lower" framed universally. The primary study concerned **52** mostly junior engineers learning one Python asynchronous library: mean scores were **50% with AI** and **67% without**, with no statistically significant average task-time gain in that study. This contextualizes comprehension risk; it does not prove Build Brief helps. https://www.anthropic.com/research/AI-assistance-coding-skills

8. "Explanation Gate is new." The research paper at https://arxiv.org/abs/2602.20206 actually tested a gate with **78 participants** and a maintenance task. Credit the source, keep an optional role-specific gate as a later enhancement and avoid claiming originality.

9. "Benchmarks of ~70% precision are enough." A lower precision may be a discovery/pivot milestone, but it is insufficient for automatic high-confidence assertions. Require mechanistically valid citations and calibrate confidence; show false-positive and abstention rates. Never set a safety assurance claim from a tiny fixture suite.

10. "The hackathon requires public repo / exactly three-minute video / new work only." These were explicitly acknowledged as unofficial notes but also used as if binding. The public organizer page confirms tracks, dates and broad theme; the participant waiver/portal must determine the exact submission constraints. https://dev.meta.ai/events/global-hackathon

11. "Ledger is proof of originality." Chronological logs can support disclosure and traceability but aren't a substitute for the organizer's eligibility rules or independent evidence that every artifact was made after kickoff. Record existing Build Brief prototype components honestly.

12. "Ghana legal eligibility equals model account availability." Ghana is not in the October 12 policy's explicit restricted-territory list, but this does not guarantee activated access or billing; an actual authenticated API smoke test is required. https://dev.meta.ai/legal/geographic-use-policy

5. Important items from the FIRST research missing or weakened in B

- **Owner approval and intent versioning:** a coding agent's extracted requirement cannot become binding policy without approval. Conflicting/superseded instructions need explicit lifecycle and effective version.
- **Static code versus runtime:** classify E1 code, E2 tests, E3 staging, E4 production; don't call "code found" a guarantee.
- **Source-grounded provenance:** every claim validates path, revision SHA, lines, snippet hash and source/test provenance; a model-supplied file reference is not sufficient.
- **Safety of imported source:** `.env` exclusions, archive traversal, symlink escape, malicious build scripts, repo prompt injection and cross-tenant access are first-order concerns.
- **Correction and recheck integrity:** generate instructions, allow another agent to implement, then check a **new commit**, not mark success from the prompt.

- **Governed, nontechnical stakeholder learning:** marketing/support/billing explanations must surface gaps and code versus official business policy distinctions.
- **A real upgrade path:** local CLI to MCP and Muse to opt-in GitHub App and later multi-agent compatibility, without pretending all agents expose identical hook semantics.
- **Existing prototype context:** user-reported earlier work exists, but the current repo is not examined. Inspect before scaffolding or asserting implementation progress.

6. Gaps BOTH agents had to strengthen

1. Explicit partial-capture indicators for failed hooks/session gaps.
2. Evidence invalidation when revisions change.
3. A measured search-coverage basis for "missing."
4. Harmless orphan hunk handling and human challenge rights.
5. Prompt injection, secret scanning, permission boundaries and minimal model transmission.
6. Versioned eval labels, realistic negative/holdout cases and no fake accuracy.
7. Learning accuracy and fairness: unmeasured ≠ low comprehension, and no worker ranking.
8. Accessible status explanations and transparent cost/latency telemetry.
9. Grounded offline/Meta outage fallback.
10. Clear "what happens now" coding-agent scope and official rule verification.

7. Final merged decision: exact priority

Hackathon P0: user-confirmed contract, commit-pinned source inspect, structured Meta adjudication with mechanical citations, honest unknowns, precise remediation and a real before/after recheck, deterministic benchmark, reproducible sample.

High-value P1: Muse Code prompt hook, reverse-change review, one quiz and one role-specific tour. Add constraint guard only after understanding hook boundary; add Comprehension Map only if real quiz data exists; add autopsy if P0 stable.

Long-term product: comprehensive onboarding paths, in-context research, architecture explorer, codebase chat with evidence, scheduled digests, read-only GitHub App, enterprise governance, provenance import, CI/gates, mobile learning and agent ecosystem. **Preserve, don't implement all at once.**

Final instruction: Use **Build_Brief_MASTER_Blueprint.md** as the *single engineering source of truth*. Don't paste two conflicting PDFs into an agent and ask it to invent a compromise. First ask the coding agent to inspect the real codebase, confirm event-specific requirements and break the validated P0 scope into testable tasks; then implement the smallest full loop.

Audited on October 27, 2026. No API credits, source code, external competition waiver or existing runtime were accessed/tested by this research output. All unverified account-specific requirements remain open.